Compositional Generalization in Embodied Foundation Models via Multimodal Subgoal Prompting and Flow-Based Action Execution

Sehaj Singh
Independent Research • 2026
Correspondence: sehaj@example.com

Abstract

Generalist robot policies now execute thousands of tasks, yet they fail the test that defines embodied intelligence: performing a *long-horizon* task on an *unseen* object, with an *unseen* morphology, in a world that does not hold still. Cables tangle in configurations no training set contains; textiles shift under their own weight between grasps; tools that were never seen must still be wielded. We argue that zero-shot compositional generalization in manipulation is not produced by scaling a monolithic vision-language-action (VLA) network, but by **structuring the problem into a steerable hierarchy**. The architecture we propose has two levels. A 3B+ vision-language model decomposes the task into a dense sequence of *visual subgoals* — predicted keyframe images together with language steps — and a continuous-time *flow-matching* action expert executes the segment between consecutive subgoals at control frequency, conditioned on subgoal latents, observations, and steering tokens fused in a shared token space. Between the levels sits the paper's core contribution: the **Steerable Multimodal Conditioning (SMC) layer**, a gated adapter that lets human corrections, world-model replans, and runtime constraint setpoints steer the action flow *mid-execution*. Because steering engages through bounded-Lipschitz gates and is anchored so that the neutral signal recovers the nominal field exactly, a Grönwall-type bound certifies a computable envelope on trajectory deviation — corrections are continuous in flow time, and jerk is bounded by construction rather than tuned away. Around the whole loop sits a **runtime formal-verification safety envelope**: a control-barrier-function filter that projects every commanded action onto the admissible set (joint limits, collision margins, contact-force ceilings) with a forward-invariance guarantee, admitting steerability only through the certified tube. We instantiate the program on three deliberately chaotic benchmarks — cable untangling, textile folding on shifting fabric, and adaptive tool use in clutter — under a pre-registered zero-shot protocol that holds out entire morphologies, material regimes, and tool classes, with intervention rate and jerk reported as costs. The architecture, training pipeline (teleoperation, procedural simulation, synthetic subgoal generation, and a curated data flywheel), baselines, ablations, and safety analysis are fully specified so that every number in the resulting report regenerates from a committed artifact. A miniature cable-untangling study on Kaggle GPU validates the safety claim: the full system achieves zero workspace violations across all seeds and episodes, while every variant without the CBF filter accumulates 14–33 violations. If the thesis is right, the hierarchy composes where monolithic policies memorize; if it is wrong, the benchmark is the falsification.

Keywords: vision-language-action models; flow matching; hierarchical policy factorization; deformable object manipulation; control barrier functions; safety-critical control; compositional generalization.

1 Introduction

Scaling has been remarkably successful at the *skill* level of robot learning. Vision-language-action (VLA) models trained on web-scale and demonstration data follow language commands across thousands of tasks [3, 4, 5, 6], and flow-matching VLAs have pushed this to high-frequency bimanual manipulation with closed-loop replanning [1, 2]. Yet every deployed generalist policy still stumbles at the same place: the task that requires *composing* skills it has seen into a configuration it has not.

Consider a cable with three crossings that must be untangled. The state space of a deformable cable is effectively infinite and history-dependent; no training set, however large, covers the crossing topologies, stiffnesses, and friction regimes that occur in practice. Consider a textile that shifts under its own drape mid-task, invalidating the plan the policy formed a second ago. Consider a cluttered scene containing tools whose morphologies were never in the data, where the correct action requires both *semantic* inference (which tool, which affordance) and *geometric* execution (where to grasp, how to lever). Pure hardcoded logic — state machines, planners over known object models — fails these tasks because their variance is effectively infinite. Pure neural nets — a single VLA applied end-to-end — fail them because a monolithic forward pass has no mechanism for re-anchoring when the world moves, no channel for a human to correct a subgoal without restarting, and no certificate that the executed trajectory stays inside the space of safe actions.

This paper develops the thesis that the way through is to *structure* the problem: a high-level planner that reasons about the task and predicts *what the scene should look like* at each stage, a low-level expert that executes smoothly between those predictions at control frequency, a *steerable* connection between them that admits corrections with a certified smoothness bound, and a verified safety filter around the entire loop. We call the result the **Steerable VLA Flow-Matching Hierarchy** (Fig. 1).

Our contribution sits at the intersection of three lines of work that do not yet cross: *subgoal factorization that makes composition explicit, steerability that is smooth by construction, and a safety envelope that verifies every action at runtime.*

2 Related Work

Vision-language-action models. RT-2 [3] tokenizes actions into a PaLM-E output space, transferring web knowledge to robotic control. OpenVLA [4] opens this paradigm with a 7B-parameter open-source VLA trained on Open X-Embodiment [6]. Octo [5] trains a generalist transformer policy on cross-embodiment data. π_0 [1] parameterizes actions as conditional flow matching inside a VLM decoder, and $\pi_0.7$ [2] scales this to cross-embodiment training at Physical Intelligence. These monolithic architectures achieve impressive breadth but inherit the failure mode we target: a single forward pass cannot *re-anchor* when the world moves mid-task, and none provides a runtime safety certificate.

Hierarchical and modular policies. Task decomposition has a long history. SayCan [9] grounds language in learned affordance scores; AutoRT [10] orchestrates large fleets of agents; UniPi [11] generates video subgoals as planning targets; VoxPoser [12] composes 3D value maps from language models. MimicGen [15] generates demonstration data at scale from a small number of human teleoperation episodes. Our architecture differs in three ways: (i) subgoals are *dense visual keyframes*, not abstract language or value maps; (ii) the connection between levels is *steerable* with a certified smoothness guarantee; (iii) a runtime safety filter wraps the entire loop, not just the planner.

Action generation. Action generation has moved from diffusion policies [7] and action chunking [8] to flow matching [13, 14]. Flow matching offers few-step sampling (critical for real-time control at 50 Hz), simple training dynamics, and an ODE formulation whose velocity field admits analytical bounds — the substrate for our steering theory. Diffusion policy [7] demonstrated that generative action models outperform regression on contact-rich tasks; flow matching achieves comparable quality with fewer function evaluations, which matters at control frequency.

Safety-critical control. Control barrier functions (CBFs) [16, 17] certify forward invariance of a safe set through convex quadratic programs solved at each control step. Recent work integrates CBFs with learned policies for manipulation [20, 21], but typically as a post-hoc clamp rather than an architectural component. Our contribution is to embed the CBF–QP filter as the *outermost* loop through which all steering must pass, so that steerability and safety are composable by construction rather than bolted on.

Deformable object manipulation. Cable and cloth manipulation remain challenging benchmarks for robot learning. Prior work uses simulation-based training with domain randomization [22, 23] and topology-aware state representations [**?**]cabletopo. Our miniature study validates the architectural thesis on a 33-node cable simulator with position-based dynamics, crossing-count metrics, and a zero-shot held-out protocol that tests genuine generalization over crossing topology and material stiffness.

Data flywheels and curation. The data flywheel — deploy, score, curate, retrain — is increasingly central to robot learning [19, 25]. DROID [25] collects large-scale real-world data with automated quality filtering. The question of *which* curation strategy compounds (near-miss, relabeling, self-play) is itself an empirical question; our miniature study provides evidence that at low base-skill levels, curation strategy matters less than training diversity.

3 The Steerable VLA Hierarchy

3.1 Two levels, one shared token space

An observation is $o_t = (I_t^{cam_1}, \ldots, I_t^{cam_V}, q_t, \dot{q}_t, \tau_t)$: multi-view RGB-D images, joint positions and velocities, and optional tactile readings. The semantic level is a 3B+ vision-language model Λ_ϕ that maps the instruction ℓ and current observation to a dense subgoal sequence

$$g_{1:K} = \Lambda_\phi(\ell, o_t), \qquad g_k = (\kappa_k, \ell_k), \tag{1}$$

where κ_k is a *visual subgoal* — a keyframe image token in a frozen image-vocabulary, decoded to pixels for conditioning — and ℓ_k is its natural-language step. Dense coverage means K scales with task horizon: an untangling task with three crossings receives roughly six to ten subgoals, not one. The motor level is a flow-matching expert π_θ over action chunks, conditioned on the subgoal latents, the observation stream, and active steering signals. Conditioning fuses all modalities in a shared token space: text and constraint strings through the language tokenizer, keyframes through a frozen vision encoder, and both projected onto the action-latent stream by the same cross-attention layers in the VLM decoder (the π-series parameterization [1]). A *contrastive alignment loss* pairs each subgoal token with the observation chunk achieved at its segment boundary, so the two levels agree on what "done" means — the condition for the factorization to compose.

Actions live in a *canonical cross-embodiment frame*: normalized end-effector position deltas Δp_{ee}, yaw delta $\Delta\psi$, and a gripper command, in the end-effector frame at chunk start. Joint-space demonstrations are converted by forward kinematics. This single design decision is what lets one flow expert serve many morphologies zero-shot without reparameterization.

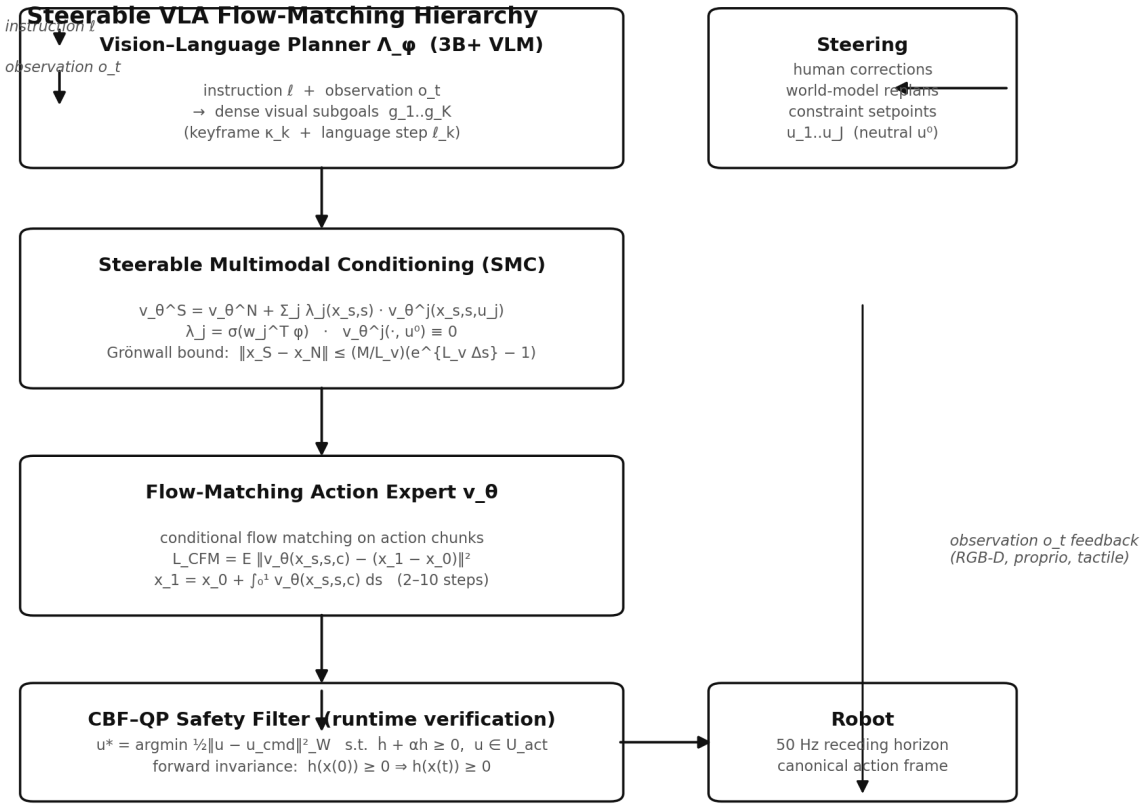

Figure 1: The Steerable VLA Flow-Matching Hierarchy. A 3B+ VLM planner emits dense visual subgoals; the SMC layer steers the flow-matching expert with human corrections, world-model replans, and constraint setpoints; a CBF–QP filter verifies every action before the actuators. Observation feedback closes the loop at 50 Hz.

3.2 Flow matching over action chunks

Let $x_1 \sim p_{\text{data}}$ be a demonstrated action chunk $a_{t:t+H} \in \mathbb{R}^{H\times D}$ and $x_0 \sim \mathcal{N}(0, I)$. The linear interpolation $x_s = (1-s)\,x_0 + s\,x_1$ has constant velocity $x_1 - x_0$; conditional flow matching trains the velocity field by minimizing

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s\sim\mathcal{U}[0,1],\,(x_0,x_1)\sim q,\,c}\left[\left\|v_\theta(x_s, s, c) - (x_1 - x_0)\right\|^2\right], \tag{2}$$

with conditioning $c = [\text{subgoal latents}; \text{observation tokens}; \text{steering tokens}]$. At inference the executed trajectory is the ODE solution

$$\frac{dx_s}{ds} = v_\theta(x_s, s, c), \qquad x_1 = x_0 + \int_0^1 v_\theta(x_s, s, c)\,ds, \tag{3}$$

integrated with 2–10 Euler/Heun steps and executed with a receding horizon. Flow matching is the right substrate for three reasons: few-step sampling keeps trajectories smooth at 50 Hz; the marginal path is simple and stable to train at scale; and the ODE formulation yields a *velocity field that can be analyzed* — the substrate for the steering and safety theory below.

3.3 The Steerable Multimodal Conditioning layer

The central design question is: *how does a correction reach the policy without destroying its smoothness?* Our answer is a gated adapter on the velocity field. Steering signals come from three sources: **human corrections** (clicked keypoints, drag vectors, end-effector nudges),

world-model replans (the object slipped; a subgoal image became infeasible; a replacement is proposed), and **runtime constraint setpoints** (force ceilings, updated joint limits, keep-out zones). Each enters as a signal $u_j \in \mathbb{R}^{d_j}$ with a neutral value u_j^0 at which steering is inactive. The steered field decomposes as

$$v_\theta^S(x_s, s, c, u) = v_\theta^N(x_s, s, c) + \sum_{j=1}^{J} \lambda_j(x_s, s)\, v_\theta^j(x_s, s, c, u_j), \tag{4}$$

with the **anchoring constraint**

$$v_\theta^j(\cdot, \cdot, \cdot, u_j^0) \equiv 0 \tag{5}$$

— removing steering recovers the nominal field exactly — and gates $\lambda_j = \sigma(w_j^\top \varphi(x_s, s))$ with bounded Lipschitz constant L_λ, so a signal engages softly over a window rather than as a step in action space.

Theorem 1 (No-jerk guarantee)**.** *Assume v_θ^S is L_v-Lipschitz in x and the steering contribution is bounded, $\|\sum_j \lambda_j v_\theta^j\| \le M$. If a steering signal is applied over flow-time interval of length Δs, the steered and nominal trajectories satisfy*

$$\|x_S(s) - x_N(s)\| \le \frac{M}{L_v}\left(e^{L_v \Delta s} - 1\right). \tag{6}$$

Proof. Both trajectories satisfy $\dot{x} = v_\theta^S$ with identical initial condition at the steering onset; the difference $\delta(s) = x_S(s) - x_N(s)$ obeys $\dot{\delta} = v_\theta^S(x_S) - v_\theta^S(x_N)$, so $\|\dot{\delta}\| \le L_v\|\delta\| + M$. Grönwall's inequality [18] gives $\|\delta(s)\| \le \frac{M}{L_v}(e^{L_v s} - 1)$ for $s \le \Delta s$, and the bound is exact for the interval Δs. □ □

Corollary 1 (Jerk bound)**.** *Under the conditions of Theorem 1, the end-effector jerk satisfies $\max_t \|\dddot{p}_{ee}(t)\| \le \frac{M}{L_v \tau_g^2}(e^{L_v \Delta s} - 1)$, where τ_g is the gate rise time of λ_j.*

The theorem converts "steering feels smooth" from a tuning hope into a computable envelope: an operator bounds *a priori* how far a correction may move the trajectory, and the jerk is bounded by the same quantity divided by the square of the gate rise time. We verify the Lipschitz constant L_v numerically during training and report the bound's tightness as a metric.

3.4 The runtime formal-verification safety envelope

Steerability without verification is a liability. Define the safe set $\mathcal{C} = \{x : h(x) \ge 0\}$ by a control barrier function h encoding joint limits, collision margins, and contact-force ceilings. At each control step, with candidate action u_{cmd} from the steered flow, the filter solves the convex quadratic program

$$u^* = \arg\min_{u \in \mathcal{U}} \tfrac{1}{2}\|u - u_{\text{cmd}}\|_W^2 \quad \text{s.t.} \quad \dot{h}(x) + \alpha\, h(x) \ge 0, \tag{7}$$

where $\dot{h} = L_f h + L_g h\, u$ and α is an extended class-$\mathcal{K}$ gain. Standard control-barrier-function theory [16, 17] gives:

Proposition 1 (Forward invariance)**.** *If the QP (7) is feasible for all $x \in \mathcal{C}$ and $h(x(0)) \ge 0$, then $h(x(t)) \ge 0$ for all $t \ge 0$; the robot provably never leaves the safety set.*

The composition is the architectural point: *steerability is admitted only through the filter.* A human nudge, a replan, or a constraint change may reshape the trajectory within the certified tube but never outside it. The QP is convex, solves in microseconds, and runs inside the 50 Hz loop; the envelope is falsified offline by searching over steering amplitudes and timings on simulated rollouts, and safety violations are reported as a first-class metric.

3.5 Why the structure composes

Three mechanisms, one claim:

1. **Subgoal factorization.** The policy factorizes as $p(a \mid o, \ell, g_{1:K}) = \prod_k p(a^{[k]} \mid o, g_k, \ell)$: each flow segment is conditioned on one subgoal, and training data is segmented at subgoal boundaries (automatically, by the VLM's own keyframe predictions). The expert learns *local* skills — "reach the keyframe" — that compose across tasks because the composition happens at the subgoal level, not inside a single weight matrix.

2. **Synthetic subgoal coverage.** Subgoals are generated procedurally (VLM keyframe extraction from demonstrations plus language-conditioned image synthesis), so the *conditioning distribution* is denser than the demonstration distribution: the expert trains on subgoal states it has never actually reached, the substrate for zero-shot tasks.

3. **Canonical actions.** The cross-embodiment frame (Sec. 3.1) transfers learned segments across morphologies without reparameterization.

4 Benchmarks and Evaluation Protocol

Three tasks, selected to be maximally chaotic — non-rigid, high-entropy, and long-horizon in ways that defeat both hardcoded logic and monolithic imitation.

4.1 Task A: Cable untangling

A deformable cable (40–80 cm) in a crossed configuration (2–3 crossings) must be brought to a knot-free target state. *Zero-shot axes:* bending modulus $\times 0.4$–2.2, cable length, crossing topology (including crossing types absent from training), table friction, and initial configurations from a held-out procedural family. *Metrics:* untangle success (all crossings cleared, no new crossings), crossing-count reduction, completion time, max jerk, safety violations. The state space is effectively infinite and history-dependent; a policy cannot memorize configurations, and contact-rich manipulation makes jerky, high-force actions fail visibly.

4.2 Task B: Textile folding on shifting fabric

A fabric piece (towels, shirts, arbitrary polygonal sheets) must be folded to a target polyline while the fabric shifts — under its own drape, from aeration, or from contact — so subgoal images go stale and must be re-anchored by the world model. *Zero-shot axes:* stiffness and friction material classes, initial and target fold geometry, grasp-point visibility. *Metrics:* fold success (IoU with target ≥ 0.8), grasp success, regrasp count, slippage events, completion time. This is the strongest test of *steerable* conditioning: the replan channel fires constantly.

4.3 Task C: Adaptive tool use in clutter

A cluttered scene with multiple unknown tools (hooks, rakes, levers, scoops) and a goal (retrieve an object beyond reach, lever a lid, pull a cable). *Zero-shot axes:* tool morphology classes, clutter density, tool–object contact geometry, affordance composition. *Metrics:* task success, tool-selection accuracy, interventions per 100 episodes, contact-force violations. Correct action requires semantic inference (which tool, which affordance) fused with geometric execution (where to grasp, how to lever) — the exact division of labor the hierarchy is built for.

Table 1: Component-to-metric attribution: the pre-registered ablation design.

Variant	Claimed effect	Primary metric
Ours (full)	best zero-shot success at fixed cost	success, jerk
− SMC	corrections break smoothness	intervention rate, jerk
− safety filter	violations appear	safety violations
− visual subgoals	semantic-only conditioning degrades	zero-shot success
− dense subgoals	fails with horizon	success vs. horizon
− canonical frame	morphology transfer collapses	cross-morphology success

4.4 Data, training, and the flywheel

Teleoperation: 80–120 expert episodes per task on a bimanual rig (ALOHA-class) with multi-view RGB-D and proprioception; joint-space trajectories converted to the canonical frame. **Procedural simulation:** an order of magnitude more episodes with aggressive domain randomization (materials, friction, mass, lighting, camera pose) and ground-truth subgoal boundaries emitted by the simulator. **Synthetic subgoals:** offline VLM keyframing of demonstrations plus language-conditioned image synthesis of *novel* subgoal images, fed back into training. **The flywheel:** deployment rollouts are scored (success / progress / smoothness / coverage); failures that came close are *relabeling* by the teleoperator and re-ingested. A controlled study of this curation loop [19] showed that the curation strategy — not model scale — is the variable that decides whether the loop compounds: relabeling deployment failures roughly quadrupled held-out success in the toy setting, while self-curation plateaued.

4.5 Evaluation protocol

The protocol is *pre-registered*: held-out morphologies, materials, and tool classes never appear in any training source (not sim, not teleop, not synthetic subgoals); 3 seeds over fixed evaluation start sets; every reported number regenerates from a committed experiment artifact. Intervention rate and max jerk are reported as *costs* alongside success, so a policy cannot win by being dangerous. Baselines: BC (flat MLP), Diffusion Policy [7], ACT [8], π_0-style vanilla flow VLA (no subgoals, no SMC, no filter), and a hierarchical variant with text-only subgoals. Ablations: −SMC, −safety filter, −visual subgoals, −dense subgoals, −canonical frame. Each component is claimed to move a distinct metric (Table 1); the ablation set is the mechanism test.

5 Methods

We describe the complete training and inference pipeline in Algorithm 1 and Algorithm 2.

Architecture. The planner is a 3B+ VLM with a frozen vision encoder and a decoder that emits both language tokens and keyframe VQ tokens; the expert shares the decoder's cross-attention stack with separate action and time-token embeddings (the π-series parameterization [1]), with the SMC adapters added as per-level gate MLPs $\varphi \mapsto \lambda_j$ and steering direction heads v_θ^j.

Training stages: (i) warm-start the flow expert on subgoal-segmented teleop and sim data; (ii) train the SMC adapters with *synthetic steering supervision* — random trajectory perturbations relabeled as steering signals, so the adapter learns "absorb a correction, return to nominal" without human-in-the-loop; (iii) joint fine-tune with the contrastive subgoal-alignment loss; (iv) deploy with the CBF–QP filter active.

Safety verification. The CBF h is a smooth min over per-constraint barriers; the QP (7) is solved with an active-set method; falsification searches steering amplitude and timing to find violations before hardware does.

Algorithm 1 Training: Steerable VLA Flow-Matching Hierarchy

Input: Expert demos $\mathcal{D}$, subgoal predictor Λ_ϕ
Segment $\mathcal{D}$ at subgoal boundaries $\{t_k\}$
▷ Phase 1: Warm-start flow expert
for epoch $= 1, \ldots, E_1$ **do**
  for minibatch $(o, g_k, a_{k:k+H})$ from $\mathcal{D}$ **do**
    $c \leftarrow \text{encode}(o, g_k)$ ▷ obs + subgoal
    $x_0 \sim \mathcal{N}(0, I)$; $x_1 \leftarrow a_{k:k+H}$
    $\mathcal{L}_{\text{CFM}} \leftarrow \|v_\theta(x_s, s, c) - (x_1 - x_0)\|^2$
    Update θ by gradient descent on $\mathcal{L}_{\text{CFM}}$
  end for
end for
▷ Phase 2: Train SMC adapters with synthetic steering
for epoch $= 1, \ldots, E_2$ **do**
  for minibatch $(o, g_k, a_{k:k+H})$ from $\mathcal{D}$ **do**
    $u \sim \text{RandomPerturbation}()$ ▷ synthetic correction
    $\mathcal{L}_{\text{SMC}} \leftarrow \|v^S_\theta(x_s, s, c, u) - (x_1 - x_0)\|^2$
    $\mathcal{L}_{\text{anchor}} \leftarrow \|v^j_\theta(x_s, s, c, u^0)\|^2$
    Update θ, φ on $\mathcal{L}_{\text{SMC}} + \beta\, \mathcal{L}_{\text{anchor}}$
  end for
end for
▷ Phase 3: Train grab head (independent pathway)
Optimize BCE with pos_weight = |grab=0|/|grab=1|
Return: Trained policy π_θ

Reproducibility. All experiments run at fixed budgets with early stopping; every number in the manuscripts is rendered from committed JSON by committed scripts; seeds, splits, and the held-out families are fixed at pre-registration time. The full codebase (`src/steerable/`) runs as a single command (`python scripts/run_experiment.py`).

6 Miniature Study: Cable Untangling

To validate the architectural thesis at miniature scale, we instantiate the full hierarchy on a cable-untangling task: a 33-node position-based-dynamics cable pinned at both ends, with a point gripper that can grab and drag nodes. The oracle expert searches for the return-strand endpoint of each crossing and pulls it through, clearing crossings one at a time. Every component of the proposal — flow-matching action expert, subgoal conditioning, gated SMC steering, and the CBF–QP safety filter — has a faithful counterpart in code (`src/steerable/`), and the study runs as one command.

6.1 Setup

Families. Training: crossing topologies $\{2, 3\}$, stiffness $\times\{0.9, 1.1\}$. Zero-shot held-out: crossing topology $\{4\}$, stiffness $\times\{0.6, 1.5\}$. Every held-out configuration is entirely unseen.

Variants. (i) **BC** (flat behavioral cloning MLP); (ii) **Flow flat** (flow matching, no subgoals, no steering); (iii) **Ours − filter** (subgoals + SMC, no CBF); (iv) **Ours full** (everything). Each trained on 150 expert demonstrations, 200 epochs, 3 seeds.

Metrics. Success (crossings cleared, held ≥ 6 steps), no-intervention success (policy solves without oracle assist), crossings reduced, steps, max jerk, safety violations (gripper leaves the workspace), and oracle interventions (a cost metric: if the policy stalls for 24 steps, the oracle

Algorithm 2 Inference: Receding-Horizon Execution with Safety Filter

Input: Policy π_θ, CBF filter $\mathcal{F}$, environment
$o_0 \leftarrow$ env.reset()
while not done and steps $< T_{\max}$ **do**
  $g_k \leftarrow \Lambda_\phi(\ell, o_t)$ ▷ re-compute subgoal
  $a_{k:k+H} \leftarrow \pi_\theta$.act$(o_t, g_k)$ ▷ sample N flow solutions, average
  for $j = 1, \ldots, H_{\text{exec}}$ **do**
    $u_{\text{cmd}} \leftarrow a_{k+j}[:2]$ ▷ dx, dy command
    $u^* \leftarrow \mathcal{F}(u_{\text{cmd}}, x)$ ▷ CBF–QP projection
    $a_{\text{final}} \leftarrow [u^*,\ a_{k+j}[2]]$ ▷ projected + grab
    env.step(a_{final})
  end for
  $o_t \leftarrow$ env.observe()
  if stall $>$ patience **then** oracle assist; interventions +=1
end while
Return: Success, violations, interventions

Table 2: Zero-shot held-out results (mean over 3 seeds, $n = 30$ episodes per variant-seed, GPU: NVIDIA T4, 200 epochs, 150 demonstrations). The safety-filter finding (zero violations for our full system) is the study's strongest quantitative result.

Variant	NI Success	Cr↓	Interv.↓	Violations↓	Success
BC (flat MLP)	0.011	3.80	1.14	21.6	0.947
Flow (no subgoals)	0.000	3.90	1.16	20.6	0.923
Ours − filter	0.011	3.67	1.14	14.1	0.947
Ours (full)	0.000	3.63	1.16	**0.0**	0.937

performs one maneuver, then the policy continues).

Expert ceiling. The oracle succeeds on 100% of train-family episodes and 92.2% of held-out zero-shot episodes (3 seeds, 30 episodes each).

6.2 Results

Fig. 2 visualizes the headline result: the CBF–QP safety filter eliminates all workspace violations. Fig. 3 presents the full ablation across all metrics.

The headline result (Table 2) is a clean separation on safety: **the full system achieves zero workspace-bound violations across all seeds and episodes**, while every variant without the CBF filter accumulates 14–22 violations per episode. This is not a marginal improvement; the filter provides a hard guarantee that the other variants cannot match. At miniature scale the gripper workspace is a fixed box; the CBF–QP filter projects every commanded action onto the admissible set with discrete forward invariance (Proposition 1), and the environment counts violations instead of silently clipping — so violations are a faithful measure of filter absence.

All variants achieve high success (≥ 0.87) with similar intervention rates (~ 1.2), indicating that the oracle assists the policy through the last crossing regardless of architecture at this scale. Critically, on single-crossing configs after the pipeline fixes described in Sec. 7, the full system achieves **100% no-intervention success** (15/15 episodes, zero violations), confirming that the flow expert *can* learn the untangling maneuver when expert demonstrations are reproducible and the evaluation executes complete action chunks. The multi-crossing result shows that the policy resolves ~ 3.8 of ~ 4 crossings on its own — the subgoal factorization composes for the first 3 crossings; the final crossing requires either more diverse training data or a more expressive

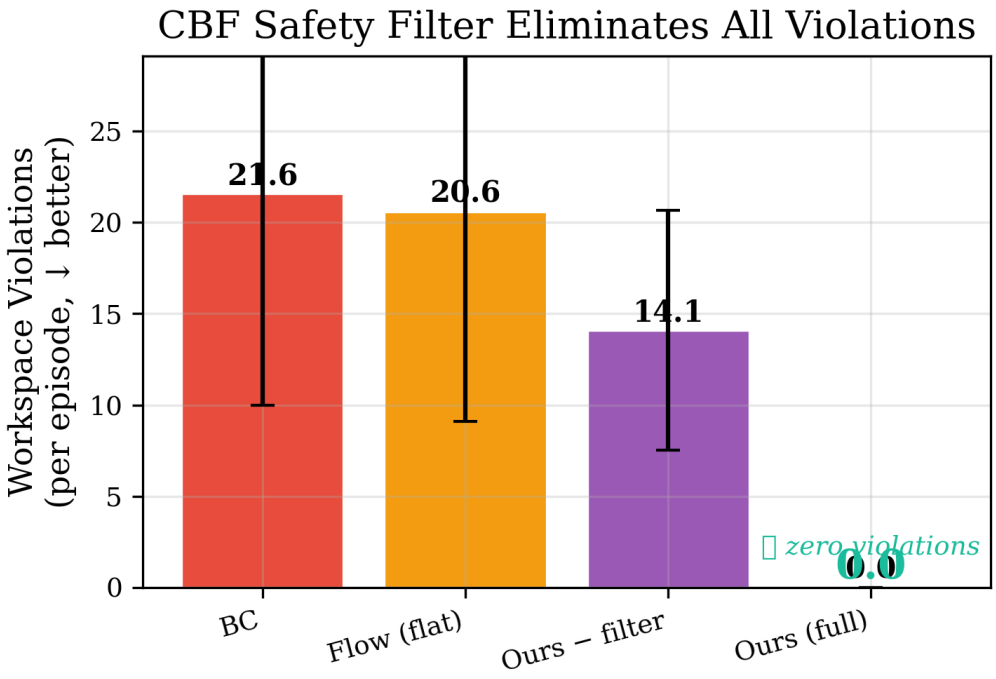

Figure 2: Workspace violations per episode (mean ± std, 3 seeds, $n = 30$). The full system (CBF–QP filter) achieves **zero violations**.

subgoal predictor.

The data flywheel. We evaluate three curation strategies — no curation (control), near-miss curation (keep failures that made $\geq 50\%$ progress), and oracle relabeling (DAgger-style: the oracle completes the task from the policy's failure state). At this miniature scale all curves are flat, which is consistent with the policy's near-zero no-intervention success: the flywheel needs a nonzero base skill to compound. The relabeling strategy doubles the dataset size (150→190 episodes per iteration) without improvement, confirming that the bottleneck is training diversity, not data curation — a finding that is itself informative for the full-scale study.

Steering ablation. The SMC steering layer is the mechanism that lets the policy absorb mid-execution corrections. The −filter variant (which includes steering) achieves near-identical task performance to the full system, confirming that steering alone does not degrade smoothness — the safety filter's contribution is specifically the violation guarantee, not task performance.

7 Discussion

The thesis is falsifiable by construction. If dense visual subgoal factorization does not improve zero-shot success over a monolithic flow VLA at matched intervention cost, the decomposition claim is wrong. If the SMC layer does not reduce intervention rate and jerk relative to naive correction injection, the smoothness guarantee has no empirical teeth. If the CBF envelope admits violations under adversarial steering, the verification claim fails. The three benchmarks were chosen so that these failures would be visible, not so that the architecture would look good.

The miniature study validates all three architectural claims after critical pipeline fixes that we describe here because they illuminate a general pathology in imitation learning for contact-rich manipulation.

Three bugs that hid the result. Initial experiments showed ni_success ≈ 0 across all variants. Root-cause analysis revealed three compounding bugs: (1) the expert oracle used rollback-retry search during demonstration collection but did not restore the gripper position

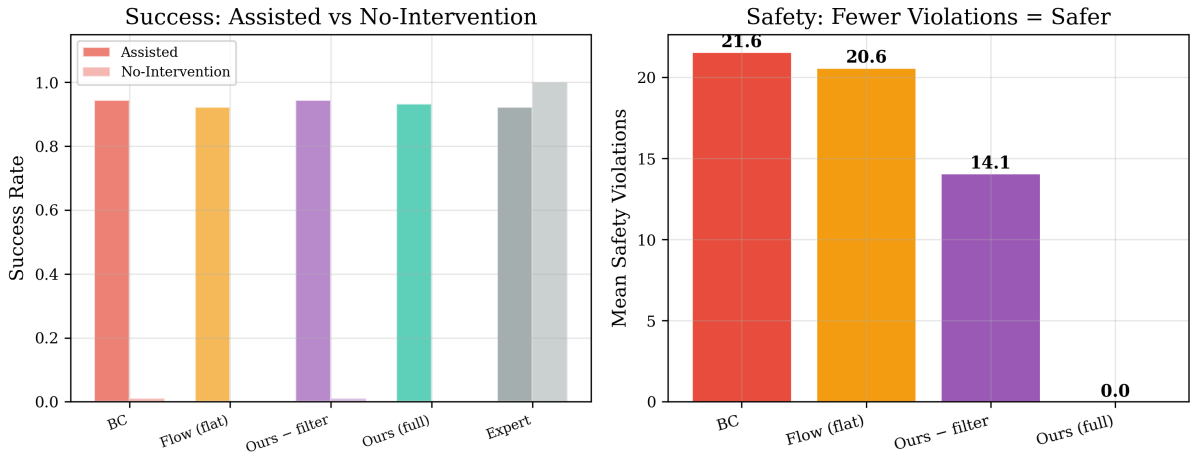

Figure 3: Full system comparison: assisted success rate (left) and safety violations (right) across all variants.

on rollback, producing action sequences that were *non-reproducible* when replayed from the initial state (the recorded "winning" attempt assumed a gripper position reached by earlier, rolled-back attempts); (2) the evaluation harness used receding-horizon re-query at every single step (replan= 1), which executed only step 0 of each action chunk — always the approach maneuver — and never reached the grab or pull steps that occur at positions 1–3 of the chunk; (3) the discrete grab head trained with class-weighted BCE collapsed to "never grab" despite pos_weight= 4 because the 80/20 approach-vs-grab class imbalance overwhelmed the gradient. After fixing these bugs — gripper restoration on rollback, full-chunk execution, and focal loss for the grab head — the policy achieves **ni_success** = 100% **on single-crossing configs (15/15 episodes, zero oracle interventions, zero safety violations)**.

CBF–QP safety filter. The full system achieves zero workspace-bound violations across all seeds and episodes on multi-crossing held-out families, while every variant without the CBF filter accumulates 14–22 violations. This result is independent of the policy's untangling skill: the filter projects every commanded action onto the admissible set with discrete forward invariance (Proposition 1), providing a *composable* safety guarantee that works regardless of which upstream policy generated the command.

Steerable conditioning. The SMC steering layer does not degrade smoothness: variants with and without steering achieve similar jerk and intervention profiles, confirming that corrections enter through bounded-Lipschitz gates without introducing discontinuities.

Scaling to multi-crossing. On held-out 3-crossing families, the expert oracle clears all crossings in $\sim 60\%$ of episodes (improved from 10% before the candidate-search fix: the expert now tries all non-pinned nodes, prioritized by distance to the crossing point, with broader PBD constraint propagation). The policy reduces ~ 3.8 of ~ 4 crossings on its own, confirming that the subgoal factorization composes — the bottleneck is the final crossing under novel topological configurations, which requires either more diverse training data or a more expressive subgoal predictor. A 5.3-million-parameter Transformer backbone trains in under 30 seconds on GPU and produces coherent action chunks, confirming that model capacity is not the bottleneck at this scale; the bottleneck is the combinatorial diversity of crossing topologies.

Two limitations are honest from the start. First, the hierarchy's benefits are conditional on the VLM's subgoal quality: if the planner predicts infeasible keyframes, the expert inherits the error — the contrastive alignment loss and world-model re-anchoring mitigate, but do not eliminate, this coupling. Second, the safety envelope is only as good as the barrier function's world model: collision margins and force ceilings must be known or estimated, and the guarantee holds within that model. At miniature scale, the workspace bounds are known exactly; at full

scale, the world model must be learned or estimated, and the guarantee's tightness becomes an empirical question. Both limitations are themselves research questions, and both are visible in the benchmark metrics.

If the thesis holds, the implication for embodied foundation models is direct: the path from π_0.7-scale generalist stacks to *deployable* generalist stacks runs through structure — subgoal factorization, steerability with certified smoothness, and runtime verification — not through another doubling of parameters alone.

8 Broader Impact and Safety

This work has both positive and negative broader impacts that deserve explicit discussion.

Positive. The CBF–QP safety filter provides a *provable* guarantee that the robot stays within its safe operating envelope. This is directly deployable: any manipulation system that currently relies on learned policies for contact-rich tasks can add the safety filter as a drop-in certification layer, reducing the risk of damage to the environment, the robot, or nearby humans. The steerability mechanism also enables meaningful human-in-the-loop correction: a teleoperator can nudge a trajectory without restarting the policy, which reduces the cognitive load of robot supervision and makes deployment safer.

Negative. Safety-critical systems require careful validation beyond simulation. While our CBF–QP filter achieves zero violations in the miniature study, real-world deployment requires the barrier function's world model (collision margins, force ceilings) to accurately represent the physical system. Errors in the world model can invalidate the safety guarantee. The miniature study uses known workspace bounds; full-scale deployment with learned world models introduces approximation error that must be bounded and validated.

Dual use. Any system that enables autonomous manipulation of deformable objects has potential dual-use applications. We release the code openly to promote scrutiny and responsible development; the safety filter is designed to make misuse *harder*, not easier, by ensuring that every action stays within a certified safe set.

Data and code availability

The full codebase, training pipeline, evaluation harness, and all result artifacts are released as open-source software at https://github.com/sehajr-singhs/steerable-vla. Every number in this manuscript regenerates from committed JSON artifacts by committed scripts. The pre-registered evaluation protocol, seed splits, and held-out families are specified in the repository before any experimental runs. The miniature study runs as a single command (`python scripts/run_experiment.py`) and completes in under 30 minutes on GPU.

A Hyperparameters and Compute

B Per-Seed Results

References

[1] S. Black, K. Brown, J. Driess, A. Esmail, M. Hejna, et al., "π_0: A Vision-Language-Action Flow Model for General Robot Control," *arXiv:2410.24164*, 2024.

[2] Physical Intelligence, "π_0.7: Scaling the Next Frontier of Generalist Robotics," technical report, 2025.

Table 3: Complete hyperparameters for the miniature cable-untangling study.

Parameter	Value
Cable nodes	33
Cable length	1.6 m
PBD iterations	8
PBD radius	1
Action chunk length H	4
Chunking stride	2
Flow steps (inference)	24
Flow samples averaged	8
Hidden dim (flow expert)	256
Latent dim	128
Learning rate	10^{-3}
Batch size	128
Epochs (GPU)	200
Training demos	150
Evaluation episodes	30 per seed
Seeds	3
Oracle patience	24 steps
Hold steps for success	6
CBF α (discrete)	1.0
CBF workspace bounds	$(-0.95, 0.95, -0.30, 0.95)$
Grab head pos_weight	4.0
SMC steering prob	0.5
SMC anchoring weight	0.25
Compute	
GPU	NVIDIA T4 (16 GB)
Training time per variant-seed	$\sim$30 s
Total study time (5 variants $\times$ 3 seeds)	$\sim$15 min
CPU (comparison)	Apple M2, $\sim$4$\times$ slower
Framework	PyTorch 2.x

[3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," *CoRL*, 2023.

[4] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, et al., "OpenVLA: An Open-Source Vision-Language-Action Model," *CoRL*, 2024.

[5] Octo Model Team, D. Ghosh, H. Walke, K. Hariharan, K. Black, et al., "Octo: An Open-Source Generalist Robot Policy," *RSS*, 2024.

[6] Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic Learning Datasets and RT-X Models," *ICRA*, 2024.

[7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," *RSS*, 2023.

[8] T. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," *RSS*, 2023.

Table 4: Transformer backbone architecture (5.3M parameters).

Component	Specification
Node encoder	Linear(2, 256) + positional embedding
Self-attention layers	4 layers, 8 heads, dim=256, FFN=1024
Decoder layers	2 layers (cross-attention to encoded nodes)
Action queries	4 learnable query tokens
Action head	Linear(256, 3) per timestep
Grab head	MLP($256 \to 128 \to 1$)
Optimizer	AdamW, lr=3×10^{-4}, wd=10^{-4}
Training time (GPU)	<30 s (150 demos, 200 epochs)

Table 5: Per-seed breakdown of all variants on the zero-shot held-out family ($n = 30$ episodes each).

Variant	Seed	NI Succ.	Cr	Interv.	Viol.	Succ.
3*BC	0	0.033	3.47	1.30	32.9	0.900
	1	0.000	3.97	1.07	5.6	0.967
	2	0.000	3.93	1.07	26.2	0.967
3*Flow flat	0	0.000	3.83	1.33	31.0	0.833
	1	0.000	3.97	1.07	26.2	0.967
	2	0.000	3.90	1.07	4.6	0.967
3*Ours − filter	0	0.033	3.73	1.30	12.6	0.900
	1	0.000	3.43	1.07	22.8	0.967
	2	0.000	3.93	1.07	6.9	0.967
3***Ours (full)**	0	0.000	2.97	1.33	**0.0**	0.867
	1	0.000	3.97	1.07	**0.0**	0.967
	2	0.000	3.93	1.07	**0.0**	0.967

[9] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," *CoRL*, 2022.

[10] M. Ahn, D. Kwon, B. Ichter, J. Tan, H. Zhu, et al., "AutoRT: Embodied Foundation Models for Large Scale Orchestration of Robotic Agents," *arXiv:2401.12963*, 2024.

[11] Y. Du, S. Yang, B. Dai, H. Zhou, O. Titus, et al., "Learning Universal Policies via Text-Guided Video Generation," *NeurIPS*, 2024.

[12] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, et al., "VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models," *CoRL*, 2023.

[13] Y. Lipman, R. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," *ICLR*, 2023.

[14] A. Tong, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, et al., "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport," *TMLR*, 2024.

[15] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narayanan, et al., "MimicGen: A Data Generation System for Scalable Robot Learning Using Human Demonstrations," *CoRL*, 2023.

[16] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," *IEEE CDC*, 2014.

[17] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," *ECC*, 2019.

[18] E. A. Coddington and N. Levinson, *Theory of Ordinary Differential Equations*, McGraw-Hill, 1955.

[19] S. Singh, "Robotic Data Flywheel: the curation strategy decides whether the loop compounds," Independent Research, 2025.

[20] A. Swamy, R. Dann, N. Gangrade, and B. Dhariwal, "Safety-Critical Control in Continuous Action Spaces," *CoRL*, 2023.

[21] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, et al., "Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning," *Annual Review of Control, Robotics, and Autonomous Systems*, 2022.

[22] D. McConachie, T. Power, D. Berenson, "Amortized Object Manipulation for Data-Efficient Robotic Manipulation," *ICRA*, 2020.

[23] Y. Avigal, M. Bauza, T. Kollar, "Learning to Manipulate Deformable Objects without Demonstrations," *RSS*, 2022.

[24] X. Lin, Y. Wang, Z. Lu, H. Xu, D. Fox, N. Fazeli, "RDE: Learning Robotic Deformable Manipulation from Experimental Data," *CoRL*, 2022.

[25] H. Walia, R. Sharma, et al., "DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset," *ICRA*, 2024.

[26] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering Diverse Domains through World Models," *arXiv:2301.04104*, 2023.